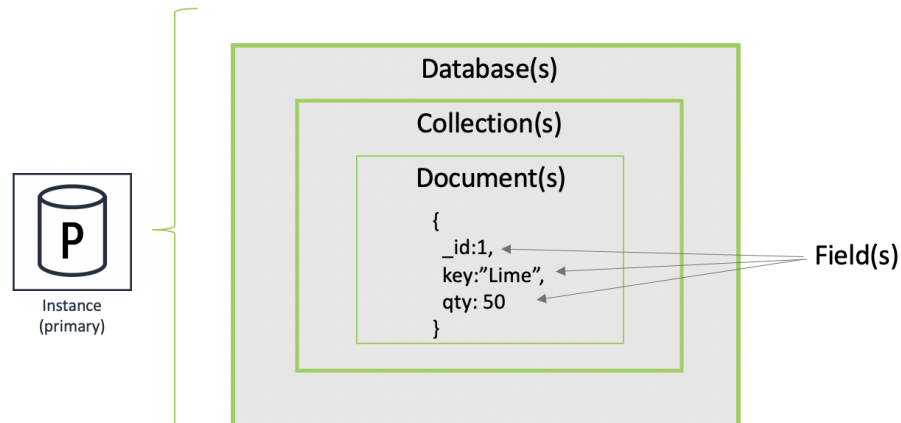


Code Talk: Coding, Scripting – what gets used where from exam point of view...

JavaScript, SQL, C#, PowerShell, Bash, cmd shell, (Azure CLI as a family name) & dotnet for starters...

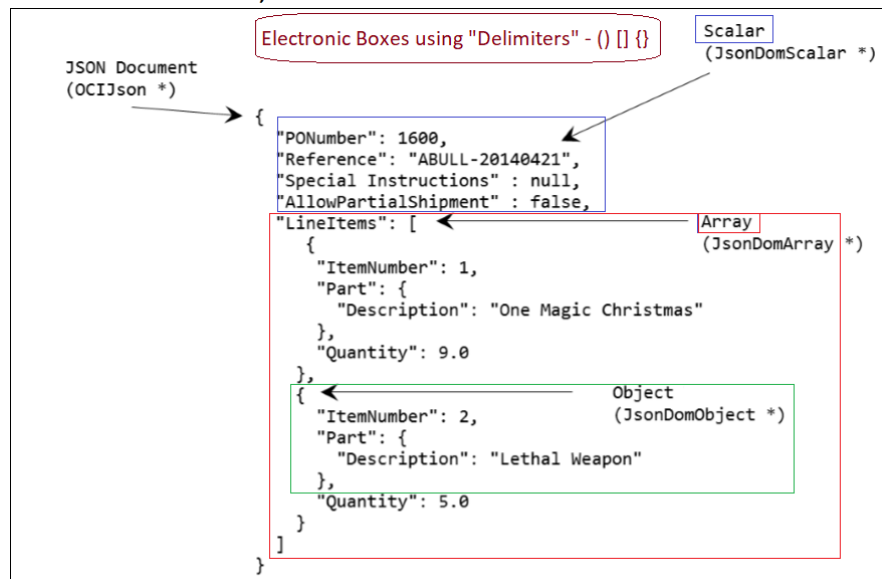
1. Creating Multi-model database – (creating data relations and working with that data)

- Document JSON & CosmosDB
- [Documents - Azure Cosmos DB REST API | Microsoft Learn](#)



id Required. It is a user settable property and is the unique name that identifies the attachment, that is, no two attachments share the same ID

c.



d.

```
select * from c IN
```

Box A

```
where
```

Box B

```
= "9.99"
```

```
{
  "id": "1",
  "Name": "UserA",
  "orders": [
    {
      "course": "Big Data",
      "price": "9.99"
    }
  ],
  "ratings": { "3": "100", "4": "200", "5": "300" }
}
```

e.

2. [CloudBlockBlob Class \(Microsoft.Azure.Storage.Blob\) - Azure for .NET Developers | Microsoft Learn](#)

```
CloudBlockBlob montanacloudBlockBlob =
cloudBlobContainer.GetBlockBlobReference(imgName);
await montanacloudBlockBlob.UploadFromFileAsync(imgFile);
```

3.

```
C#
// Create a file in your local MyDocuments folder to upload to a blob.
string localPath = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
string localFileName = "QuickStart_" + Guid.NewGuid().ToString() + ".txt";
string sourceFile = Path.Combine(localPath, localFileName);
// Write text to the file.
File.WriteAllText(sourceFile, "Hello, World!");

Console.WriteLine("Temp file = {0}", sourceFile);
Console.WriteLine("Uploading to Blob storage as blob '{0}'", localFileName);

// Get a reference to the blob address, then upload the file to the blob.
// Use the value of localFileName for the blob name.
CloudBlockBlob cloudBlockBlob = cloudBlobContainer.GetBlockBlobReference(localFileName);
await cloudBlockBlob.UploadFromFileAsync(sourceFile);
```

4.

Scripting languages for Workflow, Tasks vs. Object Oriented Programming an app...

Command	Azure CLI	Azure PowerShell
Create Resource Group	<pre>az group create --name <ResourceGroupName> -- location eastus</pre>	<pre>New-AzResourceGroup -Name <ResourceGroupName> - Location eastus</pre>

-Use PowerShell to install Azure CLI to make the AZ

(Bash) commands run

Azure PowerShell

```
Get-Command -Module Az.*
```

- cmd.exe on Windows

- Bash on Linux or macOS

-CLI commands start with az

Azure CLI

```
az webapp --help
```

-dotnet new

.NET CLI

```
dotnet build-server shutdown -h|--help
dotnet build-server -h|--help
```

-Navigate folders with cd, dir, md, ..

5.

JavaScript

Fill in the blank Q - AZ-204

Copy

```
function validateToDoItemTimestamp() {
  var context = getContext();
  var request = context.getRequest();

  // item to be created in the current operation
  var itemToCreate = request.getBody();

  // validate properties
  if (!("timestamp" in itemToCreate)) {
    var ts = new Date();
    itemToCreate["timestamp"] = ts.getTime();
  }

  // update the item that will be created
  request.setBody(itemToCreate);
}
```

```
function OrderTip() {
  var r = getContext().getRequest();

  var is=r.getBody();

  if(!("Ordertip" in i)) {
    i[tip]=0;
  }

  r.setBody(i);
}
```

6.

7. PARAMETERS:

- Specify which values you can input when deploying the resources.
- Reduce the number of parameters by using objects with multiple properties.

8. VARIABLES: (JSON vs. JavaScript)

- a. Construct values that can be used throughout your template.
- b. Define variables is not required but can simplify your template by reducing complex expressions.

```

"parameters": {
  "storageAccountType": {
    "type": "string",
    "defaultValue": "Standard_LRS",
    "allowedValues": [
      "Standard_LRS",
      "Standard_GRS",
      "Standard_ZRS",
      "Premium_LRS"
    ],
    "metadata": {
      "description": "Storage Account type"
    }
  },
  "location": {
    "type": "string",
    "defaultValue": "",
    "metadata": {
      "description": "Location for all resources."
    }
  }
},
//Declare variable or a parameter in JSON:
//This one is named location - inside the "parameters": curly brace
"variables": {
  "storageAccountName": "[concat('storage', uniquestring(resourceGroup().id))]"
},

```

inside curly brace

The curly brace delimiters create a "box" for this code block -called parameters:

JSON: JavaScript Object Notation
NOT JavaScript
var, let, const variable declarations

name

Create reusable values for deploying the resources here.

variables are used throughout your template but not intended to work with "outside" values like Azure storageAccountType or location

- c.
- d. JavaScript

```

--Psuedo code - means its all talk no execution...

//declare JavaScript variables p1, p2 and give them a value
//1 is the "argument" for p1
var p1 = 1

//2 is the "argument" for p2
var p2 = 2

//pass variable to function parameters p1, p2
function(p1, p2){
  return p1 + p2
}

```

- e.

9. FUNCTIONS:

```
JSON
{
  "functions": [
    {
      "namespace": "<namespace-for-functions>",
      "members": {
        "<function-name>": {
          "parameters": [
            {
              "name": "<parameter-name>",
              "type": "<type-of-parameter-value>"
            }
          ],
          "output": {
            "type": "<type-of-output-value>",
            "value": "<function-return-value>"
          }
        }
      }
    }
  ]
}
```

- The function can't access variables.
- The function can only use parameters that are defined in the function. When you use the [parameters function](#) within a user-defined function, you're restricted to the parameters for that function.
- The function can't call other user-defined functions.
- The function can't use the [reference function](#).
- Parameters for the function can't have default values.

a.

Birds Eye

ARM Templates

Subscription named Subscription1

Resource Group name RG1 in in North Europe Azure region.



Create a Storage account in RG1 with same location as RG1

Subscription1

 RG1 North Europe Azure
 Create a Storage

Select the ideal location value:

 North Europe
 "[resourceGroup().location]"
 "[resourceGroup().region]"
 "[RG1().location]"

```
"location": {
  "type": "string",
  "defaultValue": ,
  "metadata": {
    "description": "Location for all resources."
  }
}
```

nothing is here... empty...



Question - Answer

```
"location": {
  "type": "string",
  "defaultValue": "[resourceGroup().location]",
  "metadata": {
    "description": "Location for all resources."
  }
}
```



10.

Azure CLI

```

--Note order of syntax - cart before horse - group, appservice plan, app, deployment
has Git info out of order - incorrect

az group create --location westeurope --name myResourceGroup
az webapp create --name $webappname --resource-group myResourceGroup --plan
$webappname
az appservice plan create --name $webappname --resource-group myResourceGroup --sku
FREE

az webapp deployment source config --name $webappname --resource-group
myResourceGroup --repo-url $gitrepo --branch master --manual-integration

----- correct
az group create --location westeurope --name myResourceGroup
az appservice plan create --name $webappname --resource-group myResourceGroup --sku
FREE
deployment source vs. create source
az webapp create --name $webappname --resource-group myResourceGroup --plan
$webappname
az webapp deployment source config --name $webappname --resource-group
myResourceGroup --repo-url $gitrepo --branch master --manual-integration

----- (az webapp create source myResourceGroup --repo-url) (--
repo-url $gitrepo --branch master --manual-integration)

az group create --location westeurope --name myResourceGroup

az appservice plan create --name $webappname --resource-group myResourceGroup --sku
FREE
az webapp create --name $webappname --resource-group myResourceGroup --plan
$webappname
incorrect
az webapp create source config --name $webappname --resource-group myResourceGroup
--repo-url $gitrepo --branch master --manual-integration

----- (git clone $gitrepo) incorrect

az group create --location westeurope --name myResourceGroup

az appservice plan create --name $webappname --resource-group myResourceGroup --sku
FREE
az webapp create --name $webappname --resource-group myResourceGroup --plan
$webappname
az webapp deployment source config --name $webappname --resource-group
myResourceGroup git clone $gitrepo

```

11.

a.

Dotnet

1. [.NET Tutorial | Hello World in 5 minutes \(microsoft.com\)](#)
2. Example: dotnet new console -o MyApp -f net7.0

Tools for execution of commands through a terminal using interactive command-line prompts or a script vs. writing an app in C# or JavaScript. (Exams may or may not mention these)

cmd.exe or PowerShell on Windows - Bash on Linux or macOS

- Azure Cloud Shell (runs AZ CLI & PowerShell)
- Terminal (runs Command Prompt, AZ CLI & PowerShell in separate tabs)
- Visual Studio Code (has built in Terminal)
- Command Prompt
- PowerShell
- WSL

- Windows Subsystem for Linux (WSL) is a feature of Windows that allows developers to run a Linux environment
- [Windows Terminal SSH | Microsoft Learn](#)
 - a connectivity tool for remote sign-in (see also RDP for Windows)
- Azure Cloud Shell Connector



RHH (Rabbit Hole Handling)